Microservices

Complete preparation guide covering core principles, design patterns, communication styles, and real interview Q&A.

1. Core Principles

Microservices architecture structures an application as a collection of small, independently deployable services organized around business capabilities.

Foundational ideas

- Independent deployability: each service can be built, tested, and deployed without redeploying the entire system.
- Decentralized data management: each service owns its own data store, avoiding a shared database that couples services together.
- Polyglot persistence and technology: teams can choose the best language/database for each service's specific needs.
- Organized around business capabilities, not technical layers — e.g., an 'Orders' service rather than a shared 'Database layer'.

Monolith vs Microservices trade-offs

- Monolith: simpler to develop, test, and deploy initially; easier transactions (single database); but becomes harder to scale teams and technology choices as it grows, and a single bug can bring down the whole application.
- Microservices: independent scaling and deployment, fault isolation, technology flexibility; but adds operational complexity (distributed transactions, network reliability, service discovery, monitoring) and requires mature DevOps practices to manage well.

2. Design Patterns

These patterns solve recurring problems that emerge specifically from splitting a system into many independent services.

API Gateway

- A single entry point for clients that handles routing to backend services, authentication, rate limiting, and sometimes response aggregation.
- Shields clients from the internal service topology and lets internal services change without breaking client contracts.

Circuit Breaker

- Wraps calls to a downstream service and 'trips open' after repeated failures, immediately failing fast (or returning a fallback) instead of letting requests pile up against a struggling dependency.
- Prevents cascading failures from spreading across the whole system; after a cool-down period, the breaker allows a trial request through to test if the dependency has recovered (half-open state).

Saga Pattern

- Manages data consistency across multiple services in a distributed transaction, since traditional ACID transactions can't span independent databases.
- Choreography: each service publishes events and reacts to events from others, with no central coordinator — simple but can become hard to trace as complexity grows.
- Orchestration: a central orchestrator explicitly tells each service what to do next and handles compensating actions if a step fails — more visibility, but introduces a coordinating component.

CQRS (Command Query Responsibility Segregation)

- Separates the write model (commands that change state) from the read model (queries that return data), which can use different, independently optimized data stores.
- Useful when read and write workloads have very different scaling or query needs, but adds complexity from keeping the two models in sync (often via events).

3. Communication Between Services

How services talk to each other has major implications for coupling, latency, and resilience.

Synchronous communication

- REST over HTTP: simple, widely understood, human-readable, but adds request/response latency and tightly couples the caller to the callee's availability.
- gRPC: binary protocol over HTTP/2, offering better performance and strongly-typed contracts (via Protocol Buffers), at the cost of being less human-readable and requiring generated client/server code.

Asynchronous communication

- Message brokers/event buses like RabbitMQ or Kafka decouple producers and consumers in time — the producer doesn't need the consumer to be available.

- Enables event-driven architectures where services react to things that happened elsewhere, improving resilience and scalability, at the cost of eventual consistency and more complex debugging/tracing.

4. Interview Q&A

Key questions frequently asked in system design and backend engineering interviews.

Q: What problem does the API Gateway pattern solve?

A: Without a gateway, clients would need to know about and call every backend microservice directly, tightly coupling the client to the internal service topology and duplicating cross-cutting concerns (auth, rate limiting, logging) in every service. An API Gateway centralizes these concerns behind a single entry point, can aggregate multiple backend calls into one client-facing response, and lets internal services be restructured without breaking client contracts.

Q: Explain the Circuit Breaker pattern and its states.

A: A circuit breaker monitors calls to a dependency and has three states: Closed (requests flow normally, failures are counted), Open (after failures exceed a threshold, all requests immediately fail fast without calling the dependency, protecting it from further load and protecting the caller from long timeouts), and Half-Open (after a cool-down period, a limited number of trial requests are allowed through to check if the dependency has recovered — success closes the breaker again, failure reopens it).

Q: How do you handle distributed transactions in microservices, given each service has its own database?

A: The Saga pattern is the standard approach: instead of one atomic transaction, the operation is broken into a sequence of local transactions, each in a different service. If a step fails, previously completed steps are undone using compensating transactions (e.g., 'CancelPayment' undoes 'ChargeCard'). Choreography-based sagas use events for each service to react to; orchestration-based sagas use a central coordinator that explicitly sequences steps and triggers compensations.

Q: When would you choose asynchronous messaging over synchronous REST calls between services?

A: Choose asynchronous messaging when the caller doesn't need an immediate response, when you want to decouple service availability (the consumer can be temporarily down without blocking the producer), when you need to fan out one event to multiple consumers, or when you want to smooth out traffic spikes with a buffer. Choose synchronous REST when the caller genuinely needs an immediate result to proceed, such as validating a request before returning a response to an end user.

Q: What is CQRS and what problem does it solve?

A: CQRS separates the model used to write data (commands) from the model used to read data (queries), which can live in different, independently scaled and optimized data stores — for example, a normalized write database and a denormalized, search-optimized read database kept in sync via events. It solves the problem of a single data model being a poor fit for both heavy write validation logic and high-performance, flexible read queries, at the cost of eventual consistency between the two models.

Q: How do microservices typically discover each other's network locations?

A: Rather than hardcoding IP addresses, services register themselves with a service registry (e.g., Consul, Eureka, or the built-in discovery in Kubernetes via Services and DNS) when they start up. Callers query the registry (or rely on a client-side/server-side load balancer that queries it) to find a healthy instance to call, allowing instances to be added, removed, or rescheduled without callers needing manual reconfiguration.

Q: What are the main challenges of moving from a monolith to microservices?

A: Key challenges include correctly identifying service boundaries (a poor split creates a 'distributed monolith' with tight coupling anyway), managing distributed data consistency without cross-service transactions, handling network unreliability and partial failures gracefully, building observability (distributed tracing, centralized logging, metrics) since a single request now spans many processes, and the increased operational overhead of deploying, monitoring, and versioning many independent services.

Q: How would you version an API in a microservices architecture without breaking existing clients?

A: Common strategies include URI versioning (/v1/orders, /v2/orders), header-based versioning (Accept: application/vnd.company.v2+json), or maintaining backward-compatible changes wherever possible (additive fields, tolerant readers) so most changes don't require a new version at all. Whichever approach is chosen, the API Gateway can help route requests to the correct backend version and can support running old and new versions side-by-side during a migration period.

Q: What is the 'distributed monolith' anti-pattern, and how do you avoid it?

A: A distributed monolith happens when services are physically separated (different processes, different deployments) but remain tightly coupled — for example, sharing a database, requiring synchronized deployments, or making excessive synchronous calls that create a fragile dependency chain — so you get all the operational complexity of microservices with none of the independence benefits. Avoiding it requires genuinely decentralized data ownership, designing services around business capabilities rather than technical layers, and minimizing synchronous inter-service dependencies.

Q: How do you handle cascading failures in a microservices system?

A: Combine several techniques: circuit breakers to fail fast once a dependency is clearly struggling, timeouts on every network call so a slow dependency can't hold resources indefinitely, bulkheads (isolating resource pools like thread pools per dependency) so one failing dependency can't exhaust resources needed by others, retries with exponential backoff (and jitter) for transient failures, and graceful degradation/fallback responses so a non-critical dependency failing doesn't take down the whole user-facing request.

Pro Interview Tips

- 💡 Always be ready to explain how you'd split a specific monolith (e.g., an e-commerce app) into services — this is a very common design question.
- 💡 Know at least one real message broker (Kafka or RabbitMQ) well enough to discuss delivery guarantees (at-least-once, exactly-once, ordering).
- 💡 When asked about distributed transactions, lead with 'Saga pattern' and be ready to contrast choreography vs orchestration.
- 💡 Mention observability (tracing, centralized logging) as a first-class concern whenever you discuss microservices trade-offs.